



Python异常、模块与包

人生苦短，我学Python!



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌



目录

Contents

- ◆ 了解异常
- ◆ 异常的捕获方法
- ◆ 异常综合案例
- ◆ Python模块
- ◆ Python包

学习目标

Learning Objectives

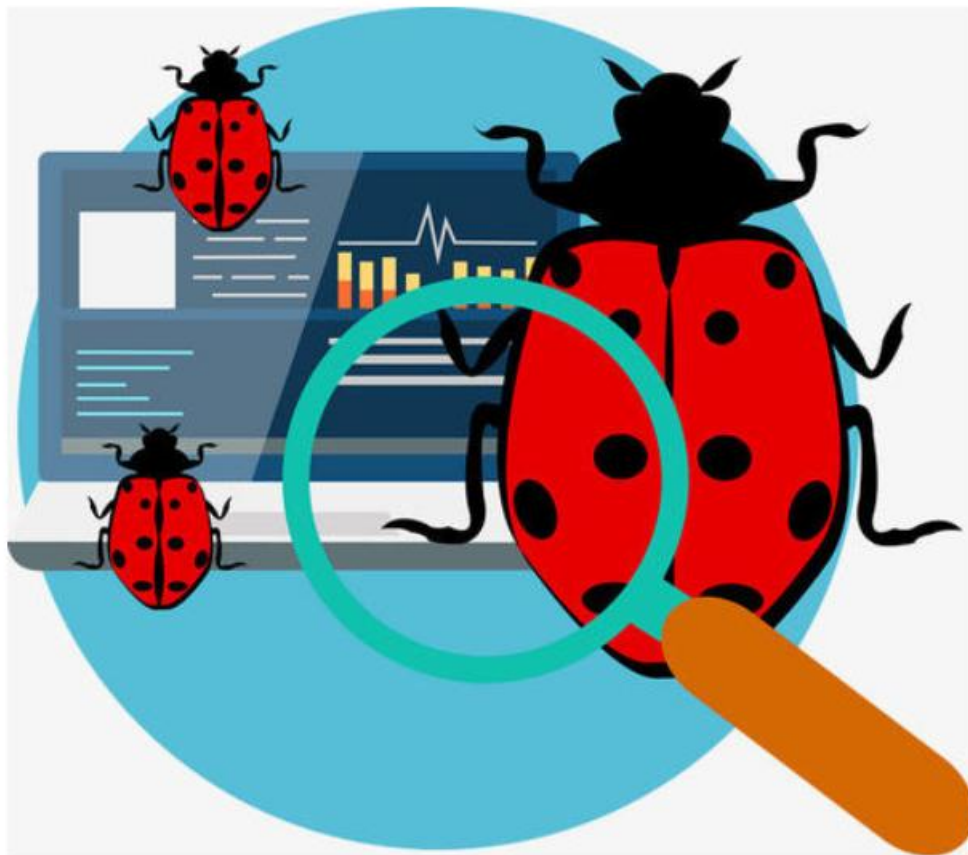
1. 了解异常的概念



了解异常

什么是异常

当检测到一个错误时，Python解释器就无法继续执行了，反而出现了一些错误的提示，这就是所谓的“异常”，也就是我们常说的BUG

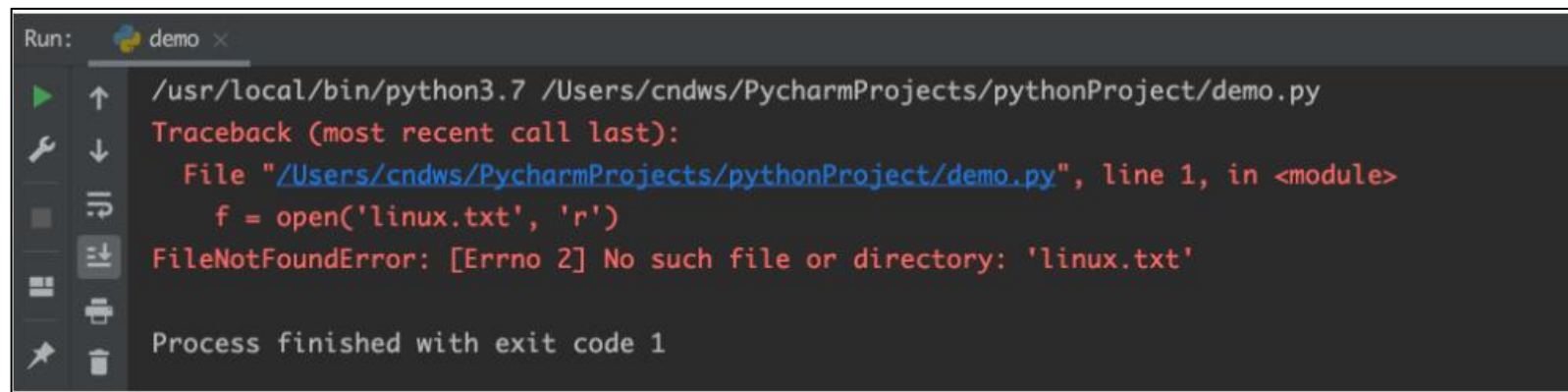


异常演示

例如：以`r`方式打开一个不存在的文件。

```
f = open('linux.txt', 'r')
```

执行结果：



```
Run: demo x
/usr/local/bin/python3.7 /Users/cndws/PycharmProjects/pythonProject/demo.py
Traceback (most recent call last):
  File "/Users/cndws/PycharmProjects/pythonProject/demo.py", line 1, in <module>
    f = open('linux.txt', 'r')
FileNotFoundError: [Errno 2] No such file or directory: 'linux.txt'

Process finished with exit code 1
```



目录

Contents

- ◆ 了解异常
- ◆ 异常的捕获方法
- ◆ 异常的传递
- ◆ Python模块
- ◆ Python包

学习目标

Learning Objectives

1. 知道为什么要捕获异常
2. 掌握捕获异常的语法格式



异常的捕获方法

为什么需要捕获异常

当我们的程序遇到了BUG，那么接下来有两种情况：

- ① 整个程序因为一个BUG停止运行
- ② 对BUG进行提醒，整个程序继续运行

显然在之前的学习中，我们所有的程序遇到BUG就会出现①的这种情况，也就是整个程序直接奔溃。

但是在真实工作中，我们肯定不能因为一个小的BUG就让整个程序全部奔溃，也就是我们希望的是达到② 的这种情况

那这里我们就需要使用到**捕获异常**

捕获常规异常

基本语法：

```
try:
    可能发生错误的代码
except:
    如果出现异常执行的代码
```

快速入门

需求：尝试以`r`模式打开文件，如果文件不存在，则以`w`方式打开。

```
try:
    f = open('linux.txt', 'r')
except:
    f = open('linux.txt', 'w')
```

捕获指定异常

基本语法：

```
try:  
    print(name)  
except NameError:  
    print('name变量名称未定义错误')
```

注意事项

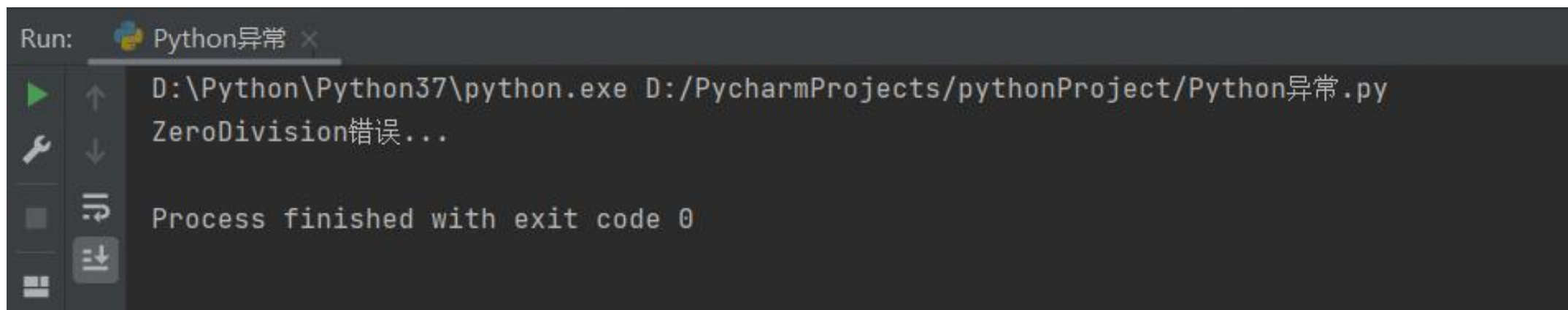
- ① 如果尝试执行的代码的异常类型和要捕获的异常类型不一致，则无法捕获异常。
- ② 一般try下方只放一行尝试执行的代码。

捕获多个异常

当捕获多个异常时，可以把要捕获的异常类型的名字，放到except 后，并使用元组的方式进行书写。

```
try:
    print(1/0)
except (NameError, ZeroDivisionError):
    print('ZeroDivision错误...')
```

执行结果：

A screenshot of the PyCharm Run console. The title bar shows 'Run: Python异常'. The console output displays the command 'D:\Python\Python37\python.exe D:/PycharmProjects/pythonProject/Python异常.py', followed by the output 'ZeroDivision错误...', and finally 'Process finished with exit code 0'.

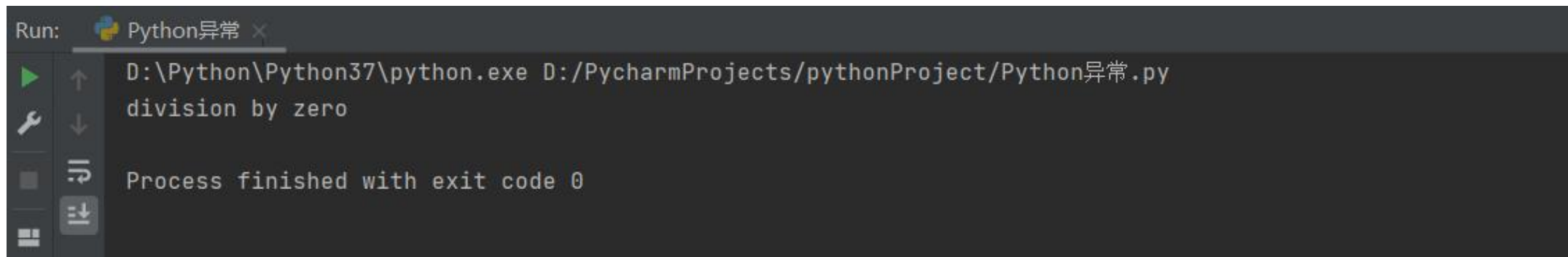
```
Run: Python异常 x
D:\Python\Python37\python.exe D:/PycharmProjects/pythonProject/Python异常.py
ZeroDivision错误...
Process finished with exit code 0
```

捕获异常并输出描述信息

基本语法:

```
try:  
    print(num)  
except (NameError, ZeroDivisionError) as e:  
    print(e)
```

执行结果:



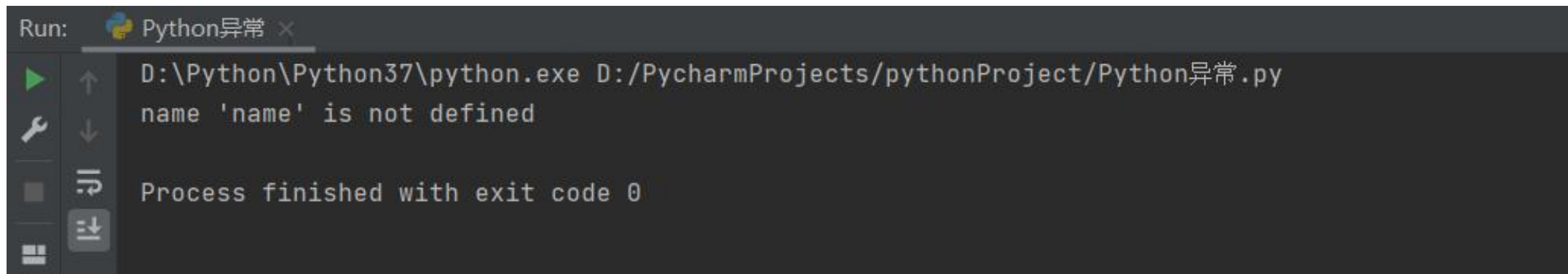
```
Run: Python异常 x  
D:\Python\Python37\python.exe D:/PycharmProjects/pythonProject/Python异常.py  
division by zero  
Process finished with exit code 0
```

捕获所有异常

基本语法:

```
try:  
    print(name)  
except Exception as e:  
    print(e)
```

执行结果:



The screenshot shows the PyCharm Run console for a file named 'Python异常.py'. The command executed is 'D:\Python\Python37\python.exe D:/PycharmProjects/pythonProject/Python异常.py'. The output shows a runtime error: 'name \'name\' is not defined'. The console also indicates that the process finished with exit code 0.

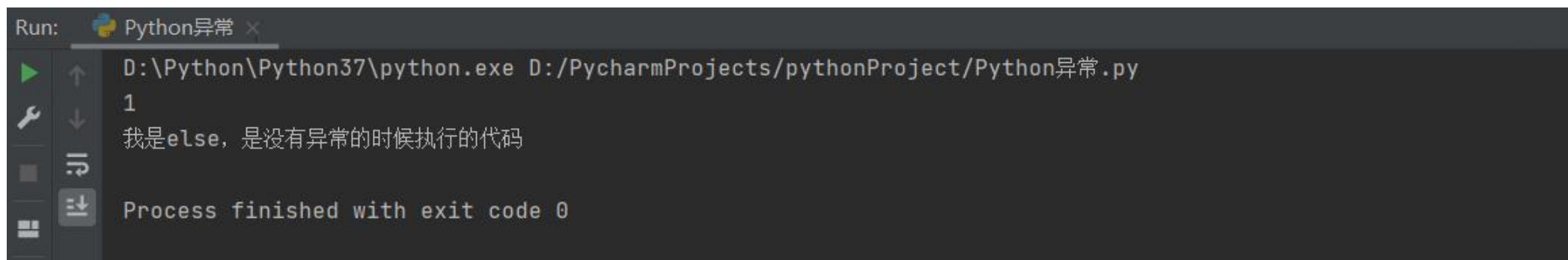
```
Run: Python异常 ×  
D:\Python\Python37\python.exe D:/PycharmProjects/pythonProject/Python异常.py  
name 'name' is not defined  
Process finished with exit code 0
```

异常else

else表示的是如果没有异常要执行的代码。

```
try:
    print(1)
except Exception as e:
    print(e)
else:
    print('我是else，是没有异常的时候执行的代码')
```

执行结果：



```
Run: Python异常 x
D:\Python\Python37\python.exe D:/PycharmProjects/pythonProject/Python异常.py
1
我是else，是没有异常的时候执行的代码
Process finished with exit code 0
```


异常的finally

finally表示的是无论是否异常都要执行的代码，例如关闭文件。

```
try:
    f = open('test.txt', 'r')
except Exception as e:
    f = open('test.txt', 'w')
else:
    print('没有异常，真开心')
finally:
    f.close()
```



目录

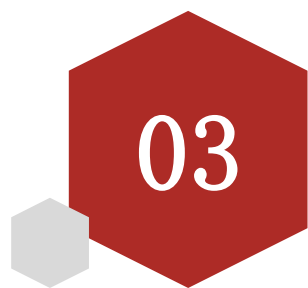
Contents

- ◆ 了解异常
- ◆ 异常的捕获方法
- ◆ 异常的传递
- ◆ Python模块
- ◆ Python包

学习目标

Learning Objectives

1. 知道异常具有传递性



异常的传递性

异常的传递

异常是具有传递性的

当函数func01中发生异常，并且没有捕获处理这个异常的时候，异常会传递到函数func02，当func02也没有捕获处理这个异常的时候main函数会捕获这个异常，这就是异常的传递性。

提示：

当所有函数都没有捕获异常的时候，程序就会报错

```
def func01(): 异常在func01中没有被捕获
    print("这是func01开始")
    num = 1 / 0
    print("这是func01结束")

    异常在func02中没有被捕获
def func02():
    print("这是func02开始")
    func01()
    print("这是func02结束")

def main(): 异常在mian中被捕获
    try:
        func02()
    except Exception as e:
        print(e)

main()
```

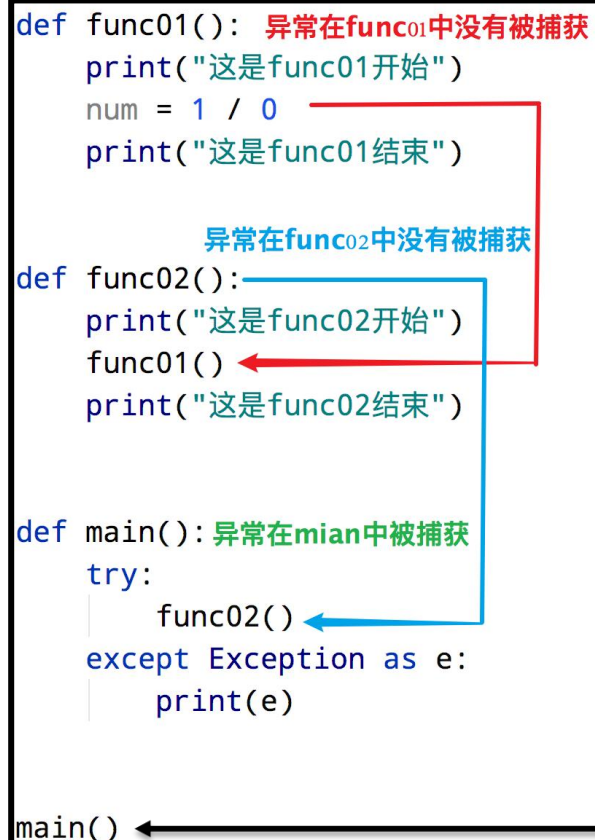
异常的传递

```
def func01(): 异常在func01中没有被捕获
    print("这是func01开始")
    num = 1 / 0
    print("这是func01结束")

    异常在func02中没有被捕获
def func02():
    print("这是func02开始")
    func01()
    print("这是func02结束")

def main(): 异常在main中被捕获
    try:
        func02()
    except Exception as e:
        print(e)

main()
```



利用异常具有**传递性的特点**，当我们想要保证程序不会因为异常崩溃的时候，就可以在**主函数**中设置异常捕获，由于无论在程序哪里发生异常，最终都会传递到**主函数**中，这样就可以确保**所有的异常都会被捕获**



目录

Contents

- ◆ 了解异常
- ◆ 异常的捕获方法
- ◆ 异常的传递
- ◆ Python模块
- ◆ Python包



学习目标

Learning Objectives

1. 了解模块
2. 导入模块
3. 制作模块



Python模块

什么是模块

Python **模块**(Module)，是一个 **Python 文件**，以 **.py 结尾**。模块能定义函数，类和变量，模块里也能包含可执行的代码。

模块的作用： python中有很多各种不同的模块，每一个模块都可以帮助我们快速的实现一些**功能**，比如实现和时间相关的功能就可以使用time模块。我们可以认为**一个模块就是一个工具包**，每一个工具包中都有各种不同的工具供我们使用进而实现各种不同的功能。



模块的导入方式

模块在使用前需要先导入 导入的方式如下:

- ❑ `import 模块名`
- ❑ `from 模块名 import 功能名`
- ❑ `from 模块名 import *`
- ❑ `import 模块名 as 别名`
- ❑ `from 模块名 import 功能名 as 别名`

import 模块名

基本语法:

```
import 模块名  
import 模块名1, 模块名2
```

模块名. 功能名()

案例：导入time模块

```
# 导入时间模块  
import time  
  
print("开始")  
# 让程序睡眠1秒(阻塞)  
time.sleep(1)  
print("结束")
```

from 模块名 import 功能名

基本语法:

```
from 模块名 import 功能名
```

功能名()

案例：导入time模块中的sleep方法

```
# 导入时间模块中的sleep方法
from time import sleep

print("开始")
# 让程序睡眠1秒(阻塞)
sleep(1)
print("结束")
```

```
from 模块名 import *
```

基本语法:

```
from 模块名 import *
```

功能名()

案例：导入time模块中的所有的方法

```
# 导入时间模块中的所有的方法
```

```
from time import *
```

```
print("开始")
```

```
# 让程序睡眠1秒(阻塞)
```

```
sleep(1)
```

```
print("结束")
```

as定义别名

基本语法:

模块定义别名

```
import 模块名 as 别名
```

功能定义别名

```
from 模块名 import 功能 as 别名
```

案例:

模块别名

```
import time as tt
```

```
tt.sleep(2)
```

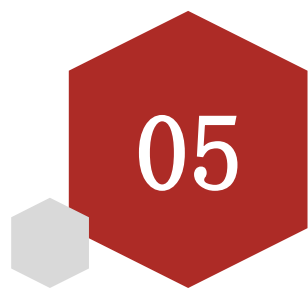
```
print('hello')
```

功能别名

```
from time import sleep as sl
```

```
sl(2)
```

```
print('hello')
```

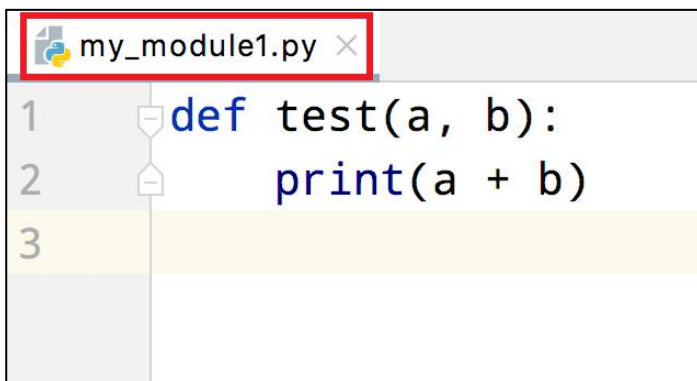


自定义模块

制作自定义模块

Python中已经帮我们实现了很多的模块。不过有时候我们需要一些个性化的模块，这里就可以通过自定义模块实现，也就是自己制作一个模块

案例：新建一个Python文件，命名为my_module1.py，并定义test函数



```
1 def test(a, b):  
2     print(a + b)  
3
```



```
1 import my_module1  
2  
3 my_module1.test(10, 20)  
4  
5  
6
```

注意：

每个Python文件都可以作为一个模块，模块的名字就是文件的名字。也就是说自定义模块名必须要符合标识符命名规则

测试模块

在实际开发中，当一个开发人员编写完一个模块后，为了让模块能够在项目中达到想要的效果，这个开发人员会自行在py文件中添加一些**测试信息**，例如，在my_module1.py文件中添加**测试代码**test(1, 1)

```
def test(a, b):  
    print(a + b)  
  
test(1, 1)
```

问题:

此时，无论是当前文件，还是其他已经导入了该模块的文件，在运行的时候都会**自动执行`test`函数的调用**

解决方案:

```
def test(a, b):  
    print(a + b)  
  
# 只在当前文件中调用该函数，其他导入的文件内不符合该条件，则不执行test函数调用  
if __name__ == '__main__':  
    test(1, 1)
```

注意事项

```
# 模块1代码
def my_test(a, b):
    print(a + b)

# 模块2代码
def my_test(a, b):
    print(a - b)

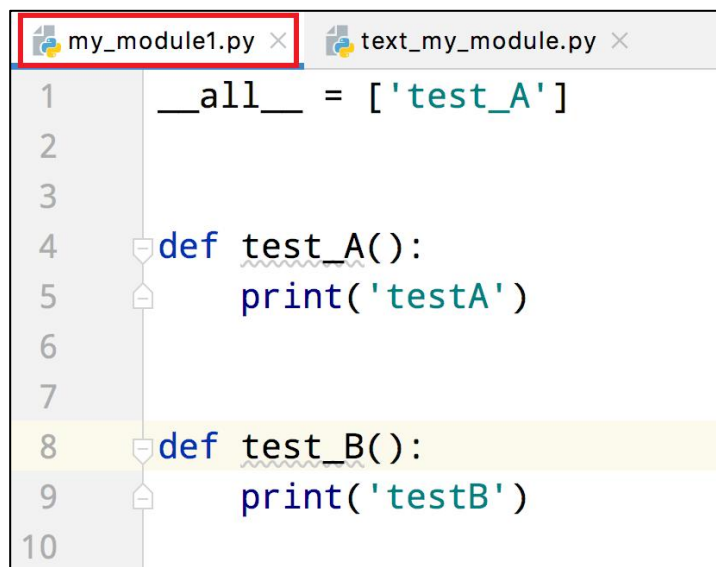
# 导入模块和调用功能代码
from my_module1 import my_test
from my_module2 import my_test

# my_test函数是模块2中的函数
my_test(1, 1)
```

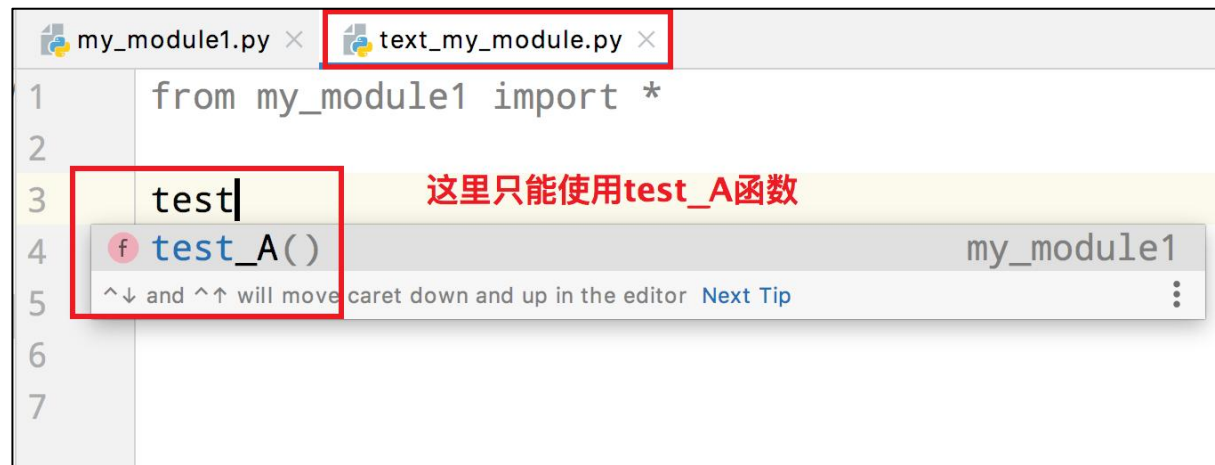
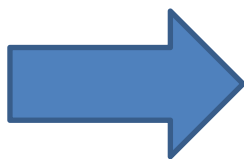
注意事项：当**导入多个模块**的时候，且模块内有**同名功能**。当调用这个同名功能的时候，调用到的是**后面导入的模块的功能**

__all__

如果一个模块文件中有`\_\_all\_\_`变量，当使用`from xxx import \*`导入时，只能导入这个列表中的元素



```
1  __all__ = ['test_A']
2
3
4  def test_A():
5      print('testA')
6
7
8  def test_B():
9      print('testB')
10
```



```
1  from my_module1 import *
2
3  test|
4  test_A()
5
6
7
```

这里只能使用test_A函数



目录

Contents

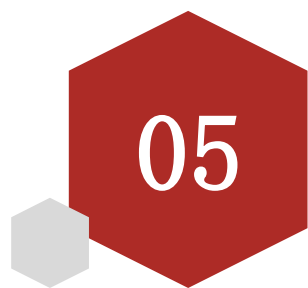
- ◆ 了解异常
- ◆ 异常的捕获方法
- ◆ 异常的传递
- ◆ Python模块
- ◆ Python包



学习目标

Learning Objectives

1. 知道如何制作包

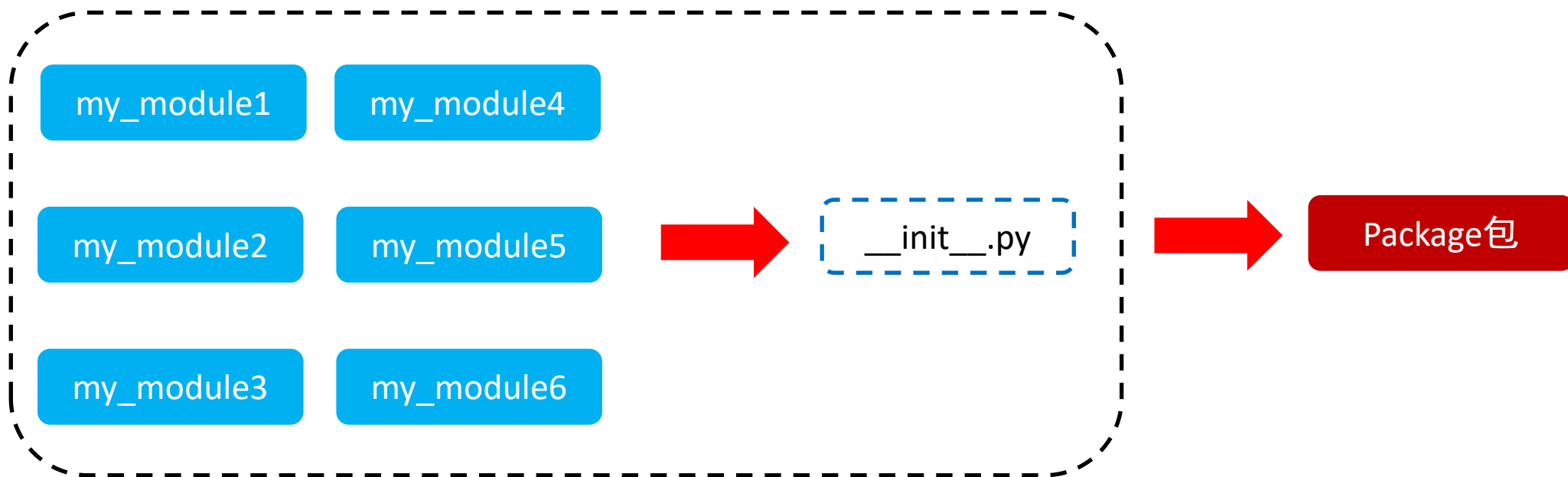


Python包

什么是Python包

从物理上看，包就是一个文件夹，在该文件夹下包含了一个 `__init__.py` 文件，该文件夹可用于包含多个模块文件

从逻辑上看，包的本质依然是模块



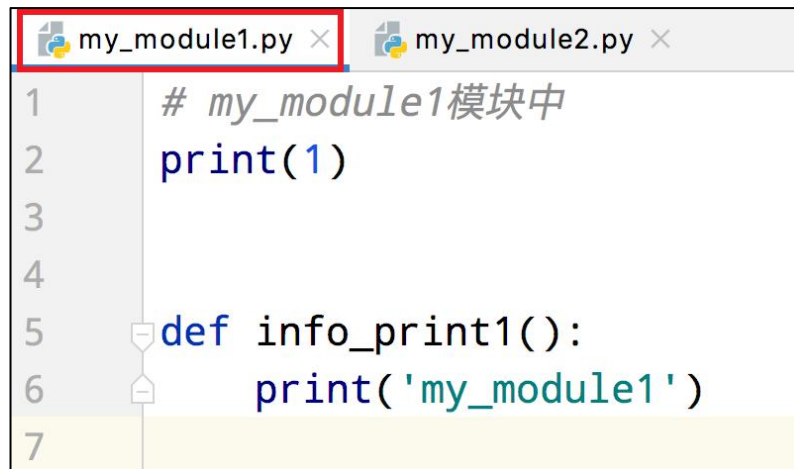
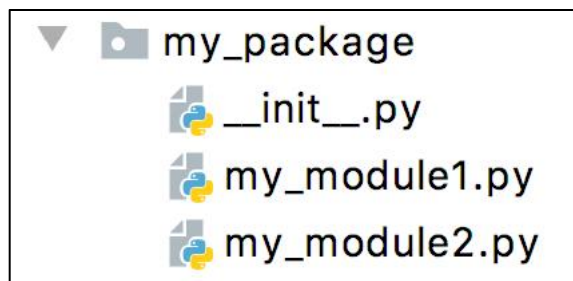
包的作用：

当我们的模块文件越来越多时，包可以帮助我们管理这些模块，包的作用就是包含多个模块，但包的本质依然是模块

快速入门

步骤如下:

- ① 新建包`my\_package`
- ② 新建包内模块: `my\_module1` 和 `my\_module2`
- ③ 模块内代码如下



Pycharm中的基本步骤:

[New] → [Python Package] → 输入包名 → [OK] → 新建功能模块(有联系的模块)

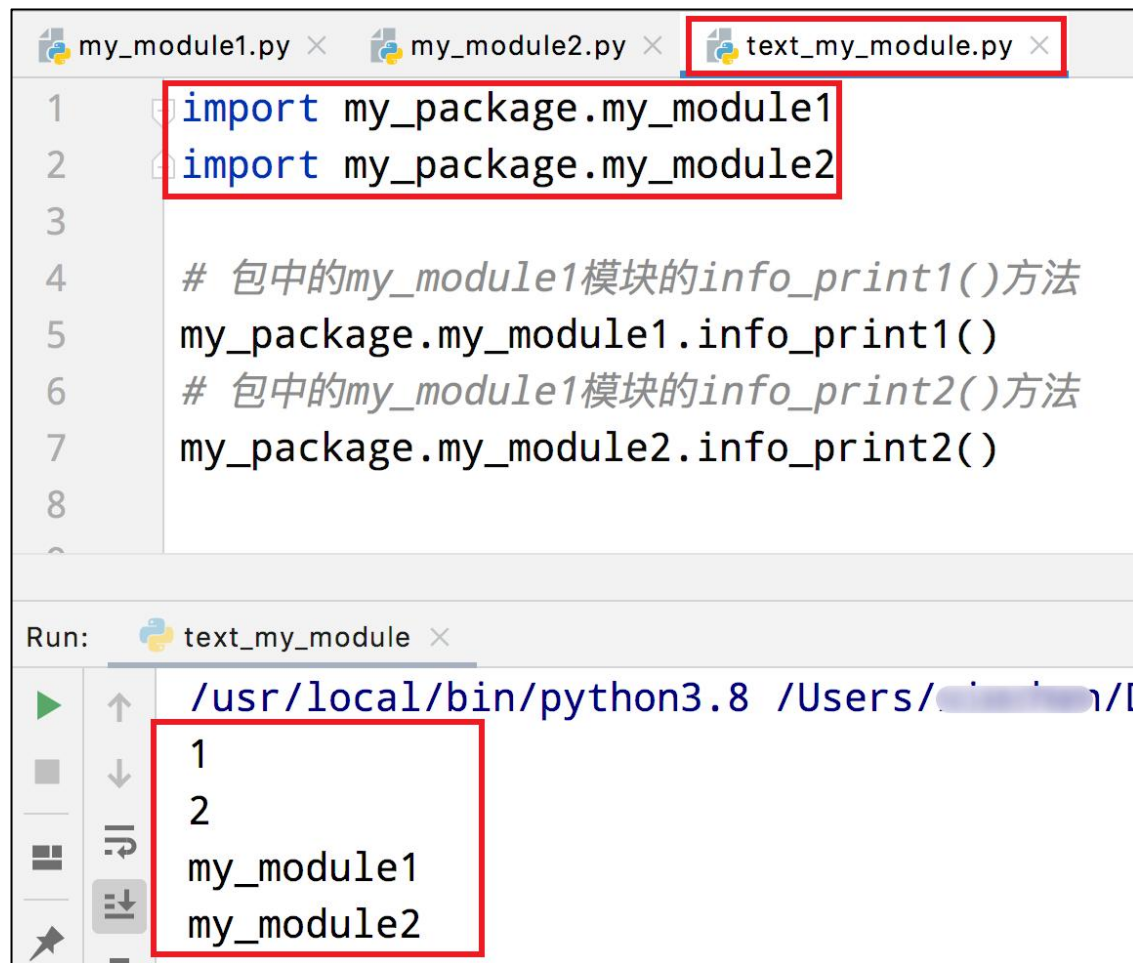
注意: 新建包后, 包内部会自动创建`\_\_init\_\_.py`文件, 这个文件控制着包的导入行为

导入包

方式一:

```
import 包名.模块名
```

```
包名.模块名.目标
```



The screenshot shows a Python IDE with three tabs: my_module1.py, my_module2.py, and text_my_module.py. The text_my_module.py tab is active and contains the following code:

```
1 import my_package.my_module1
2 import my_package.my_module2
3
4 # 包中的my_module1模块的info_print1()方法
5 my_package.my_module1.info_print1()
6 # 包中的my_module1模块的info_print2()方法
7 my_package.my_module2.info_print2()
8
```

Below the code editor, the Run console shows the command: `/usr/local/bin/python3.8 /Users/.../D`. The output of the program is displayed in a box:

```
1
2
my_module1
my_module2
```

导入包

方式二:

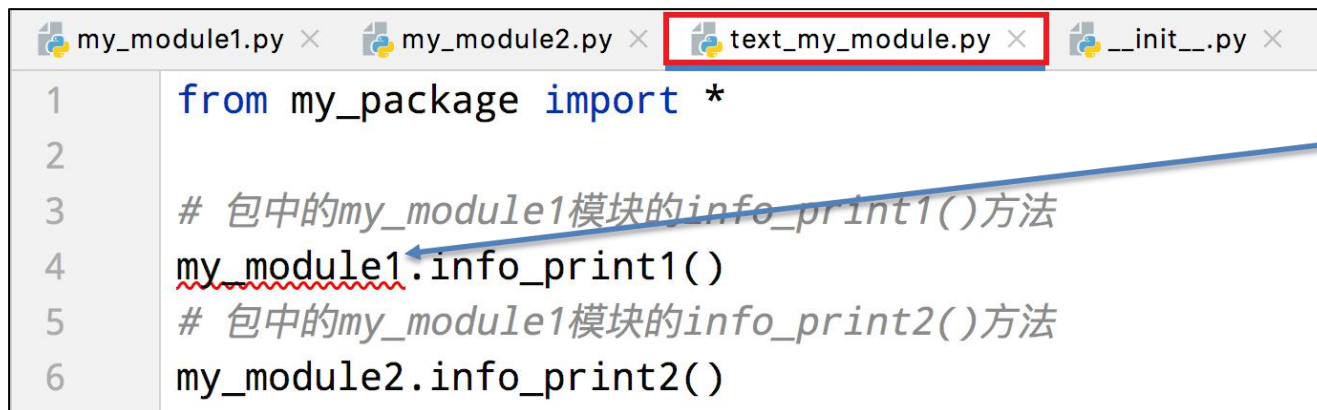
注意: 必须在`\_\_init\_\_.py`文件中添加`\_\_all\_\_ = []`，控制允许导入的模块列表

```
from 包名 import  
*  
模块名.目标
```



```
1 # 包中的__all__和模块中的__all__一样有着控制的功能  
2 __all__ = ["my_module2"]  
3 |
```

包中可以用的模块的名字

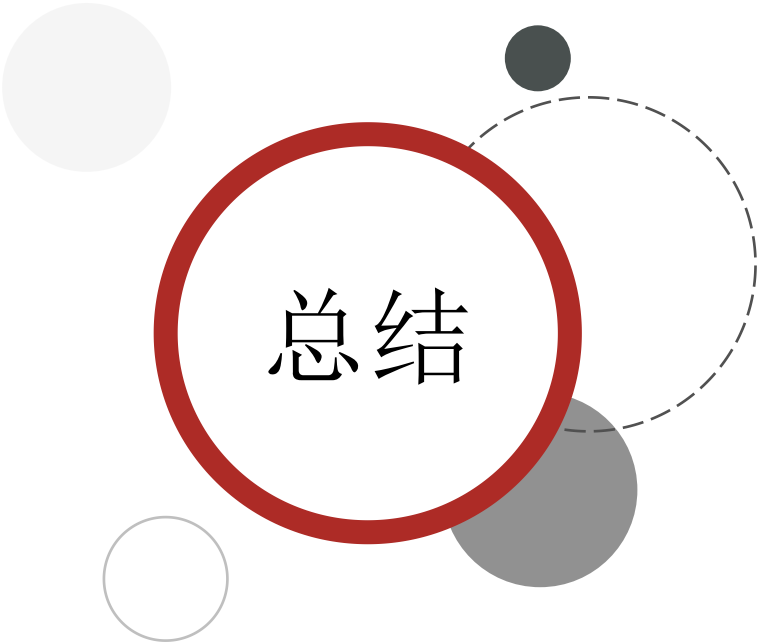


```
1 from my_package import *  
2  
3 # 包中的my_module1模块的info_print1()方法  
4 my_module1.info_print1()  
5 # 包中的my_module1模块的info_print2()方法  
6 my_module2.info_print2()
```

my_module1报红证明不可用

注意:

__all__针对的是`from ... import \*`这种方式
对`import xxx`这种方式无效



总结

•导入模块方法

- import 模块名
- from 模块名 import 目标
- from 模块名 import *

•导入包

- import 包名.模块名
- from 包名 import *
- __all__ = [] : 允许导入的模块或功能列表



传智教育旗下高端IT教育品牌